

Deelrapport 3 — Studenttool-JS (lees-alleen)

Fable 5-review, lens: architectuur & codekwaliteit, diepgang studenttool-JS. 2026-07-05.

Algehele structuur

`werkblad.js` (4496 regels) is een monoliet in **globale scope**: het bestand is ingesprongen alsof er een IIFE omheen staat, maar die is er niet (alleen een mini-IIFE op r4143–4146). Alle ~48 mutabele `var`'s en 97 functiedeclaraties zijn dus window-globals, en `matcher.browser.js` (eveneens unwrapped, ~42 globale functies) deelt diezelfde namespace — met één aantoonbare naam-botsing als gevolg. Alleen `verankering.js` is netjes als IIFE-module gebouwd. De matcher zelf is logisch het sterkste deel (geïsoleerd, node-testbaar, doordacht gedocumenteerd); de zwakte zit in de compositie: drie scripts, één namespace, en een verankering die op empirische pixel-fudges en timing-slaapjes leunt.

Bevindingen

1. Structuur / globale staat

1.1 — HOOG. Naam-botsing `treesEqual`: browser draait andere matcher-code dan het test-harnas.

(a) `matcher.browser.js:523` (`function treesEqual(a, b)` — multiset-vergelijking op matcher-nodes) vs `werkblad.js:1174` (`function treesEqual(a, b)` — MathJSON-array-vergelijking). Beide bestanden zijn plain scripts zonder IIFE; `werkblad.js` laadt als laatste (`werkblad.html:283/287`) en **overschrijft de globale binding**.

(b) De matcher roept `treesEqual` runtime aan op `matcher.browser.js:387` (`alignTarget`, "wegstrepen"-pass), `:488` (`diffPath`), `:534/:540` (eigen recursie!) en `:1121` (`pickChangedArg`). Die lookups lopen via de globale scope → in de browser krijgen ze `werkblad.js`'s versie. Op matcher-nodes (`{op,args,raw}`, geen arrays) valt die door naar `JSON.stringify(a) === JSON.stringify(b)` (`werkblad.js:1181`) — dat is **volgorde-gevoelig**, terwijl de matcher-kop expliciet "VOLGORDE-ONAFHANKELIJKE vergelijking binnen commutatieve operatoren" belooft.

(c) Faalscenario: student herordent termen in een Add/Multiply (didactisch toegestaan) → wegstrepen faalt in de browser maar slaagt in Node → verkeerd mathblock-label. Dit is exact de bug-klasse die spoor 4 / de B4-fix achtervolgde. Het harnas (451/451 groen) test de matcher in een geïsoleerde vm-context (`load_matcher.js:24–29`) en ziet deze botsing per definitie nooit.

Ook `parseDuoText` botst (`matcher.browser.js:63` vs `werkblad.js:417`); daar zijn geen interne matcher-aanroepen, dus vandaag onschadelijk — maar het is dezelfde tijdbom.

1.2 — HOOG (architectuur). Geen modulescheiding; ~48 mutabele globals.

(a) `werkblad.js:64–77` (`activeLineIndex`, `mfRef`, `currentOpgave`, `currentStep`, `remainingHoog/Laag`, `resolvedBlocks`, `lfBlocked`, ...), `:869–870` (`currentTree`, `nodeMap`), `:1748–1750`, `:2097–2099` (`atomToMathblock`, `atomRanges`), `:3163–3167` (`build-slot/throttle`), plus DOM-refs

r4088–4450. (b) Kernstaat (`currentTree` , `nodeMap` , `currentStep` , `resolvedBlocks`) wordt op minstens vier plekken gemuteerd (initTreeEngine r873, doLF r3721/3740, updateStepTracking r1721-1731, applyCorrectChanges r1648). (c) Elke nieuwe feature of extern script kan elke variabele stilletjes kapotschrijven; er is geen enkel afdwingbaar invariant. (d) Hoog als structureel risico, ook al "werkt het nu".

1.3 — MIDDEN . Dode code die live code dupliceert.

(a) `werkblad.js:1606` `applyCorrectChanges` — wordt **nergens** aangeroepen (STATUS.md r289 bevestigt), maar bevat een tweede, licht afwijkende implementatie van dezelfde nodeMap-chirurgie die doLF inline doet (r3689–3755). Ook `pinpointFromTrees` (r1308) heeft geen aanroeper, en `atomCharMap` (r2097) is gemarkeerd "Legacy - not used anymore". (c) Wie de nodeMap-logica fixt, fixt hem voorspelbaar op één van de twee plekken. (d) Midden.

1.4 — MIDDEN . Copy-paste-parsers: zes-notaties-probleem zit in de code gebakken.

(a) `werkblad.js:269–322` (`latexToMathJs`) en `:328–373` (`latexToDuo`) zijn ~45 regels identiek op één gedragsverschil na (`:` behouden). `parseDuoText` bestaat drie keer: `werkblad.js:417` , `matcher.browser.js:63` (string-vorm) en `:96` (typed-vorm); de matcher-kop verwijst bovendien naar verouderde regelnummers ("werkblad.js regels 327-353"). (c) Elke breuk-notatie-fix moet op 2–3 plekken tegelijk (dat is aantoonbaar al misgegaan: de v156/v157-hacks). INVENTARISATIE-doc erkent dit; de consolidatie (FASE 2) is uitgesteld. (d) Midden.

2. Verankering — fragiliteit

2.1 — HOOG . Empirische pixel-fudges als fundament.

(a) `verankering.js:36–43` : `GLOBAL_DELTA_CORR = { x: -1, y: -1 }` en `DEPTH_SIZE_CORR = { 0: {dw:3,dh:4}, 1:{dw:2,dh:6}, 2:{dw:5,dh:0}, 3:{dw:6,dh:-3}, 4:{dw:2,dh:-4} }` — een per-diepte gemeten correctietabel bij `REF_FONT_SIZE = 28` . (c) Elke MathLive-versie, font- of CSS-wijziging invalideert de tabel stilletjes; boxen verschuiven px-gewijs zonder foutmelding. drawBox moet zelfs expliciet weten wanneer de fudge NIET mag (`depth == null` , r125) — de aanroepers op `werkblad.js:4031–4033` coderen dat per box-soort met de hand. (d) Hoog.

2.2 — HOOG . `computeDelta` hangt aan MathLive-internals in de shadow DOM.

(a) `verankering.js:82–95` : `sr.querySelectorAll('.ML__cmr')` , glyph-tekst vergelijken met het offset-latex-cijfer, en accepteren bij `bestDist < 8` (magic). (c) MathLive hernoemt `.ML__cmr` of wijzigt glyph-rendering → geen match → stille fallback naar de nudge (r104), boxen potentieel verkeerd zonder signaal. Versie-pinning op 0.110.0 is het enige vangnet. Het doc-spoor "delta.y=20.23" (STATUS.md, spoor 2) laat zien dat een verkeerd gekozen glyph precies dit faalbeeld gaf. (d) Hoog.

2.3 — MIDDEN . Anker-koppeling = sequentiële tekst-matching, faalt stil.

(a) `verankering.js:294–317` `anchorOffsets` : loopt offsets en tokens parallel af met een monotoon oplopende `ti` , matcht op genormaliseerde latex-strings (`_norm` , r292). `genLatexTokens` emit bij onbekende operator letterlijk `'?'+opNaam+'?'` (r237). (c) Eén serialisatie-verschil tussen MathLive-offset-latex en de zelfgebouwde tokenstroom (bv. `\cdot` vs `\times` , MixedNumber-vormen) → alle volgende offsets schuiven één token op → kaders om de verkeerde subexpressie, zonder waarschuwing. (d) Midden.

2.4 — MIDDEN . `position:fixed` -boxen zonder scroll/resize-herteken.

(a) `verankering.js:144` (`div.style.position = 'fixed'`); geen enkele scroll/resize-listener in

werkblad.js of verankering.js (grep bevestigt: alleen `scrollIntoView` r629). (c) Scrollen of venster-resizen laat de boxen los van de wiskunde zweven tot een volgende render (STATUS.md geval 8 bevestigt). (d) Midden (bekend en gedocumenteerd, maar structureel onopgelost).

2.5 — LAAG. `maxProbe = 200` (`verankering.js:47`): expressies met >200 offsets worden stil afgekapt; ook `anchorOffsets` / `anchorStudentOffsets` (r294 vs r320) zijn regel-voor-regel gedupliceerd op het label-object na.

3. Matcher — deling/synchronisatie/determinisme

3.1 — GOED (ter balans): het harnas laadt byte-identiek dezelfde matcher.

`load_matcher.js:20-29` evalueert `werkblad/matcher.browser.js` in een vm-context — géén tweede node-versie, geen duplicaat. mathjs is aan beide kanten 12.4.1 (`werkblad.html:277`, `package.json`). MAAR: zie 1.1 — de isolatie die dit mogelijk maakt, verbergt tegelijk de globale-scope-botsing die in de browser wél actief is.

3.2 — MIDDEN. Determinisme hangt op tie-breaks en een data-invariant die niet wordt afgedwongen.

(a) `matcher.browser.js:420-441`: kandidaten worden gesorteerd op skelet+waarde, daarna `return sa[rest[0]]` — bij gelijke score wint de positie in de student-invoer. De twin-guard (r404) slaat waarde-matching alleen over bij exact gelijke waarde als het target. (c) Twee onopgeloste blokken met gelijk skelet én gelijke waarde in dezelfde step → koppeling wordt positioneel gegokt. STATUS.md r39-41 zegt dat tweelingen "op verschillende steps zitten" — dat is een eigenschap van de huidige 26 opgaven, nergens een gecontroleerde invariant (geen assert in `checkStep`, geen `authortool-check`). Eén nieuwe opgave kan het breken; `repro_tweeling.js:50-52` test het adversaire geval wel, maar alleen voor 511_010. (d) Midden.

3.3 — MIDDEN. `_grpCounter` is globale mutabele parse-staat (`matcher.browser.js:146`): group-id's lopen over parse-aanroepen heen op. `grp` is netjes non-enumerable gemaakt (r155-160), maar functies als `findGroupInTree` vergelijken op `grp`-waarde — elke toekomstige cache/hergebruik van geparste bomen over aanroepen heen introduceert stille cross-talk. Laag-midden.

4. Races (concreet)

4.1 — MIDDEN. Dubbel-LF binnen 200 ms → onterechte foutmarkering.

(a) `werkblad.js:3842` zet `mfRef=mf` synchroon, maar de waarde komt pas in de `setTimeout(...,200)` op r3843-3852; `doLF` heeft geen re-entrancy-guard (r3628-3636) en `getEditorLatex()` (r777) leest `mfRef`. (c) Tweede LF-klik (of Enter-repeat) binnen 200 ms → lege latex → `evaluateExpression('')` = null → `isCorrect` false → rood kruis + "Fout!" op een correcte regel. (d) Midden.

4.2 — MIDDEN. Init-choreografie op magic sleeps in plaats van op events.

(a) `setTimeout(...,100)` r713/r3789 (setValue read-only velden), `setTimeout(...,200)` r743/r3843 (setValue + listener-koppeling + focus), `setTimeout(mlGo,100)` r4431, retry `setTimeout(...,500)` r2074. De commentaren documenteren zelf drie eerdere input-cascade-loops en een focus-race ("de DERDE plek met dit patroon", r3798). (c) Op een traag apparaat of bij een MathLive-upgrade verschuiven de venstertjes en komt dezelfde klasse loops/races terug; niets wacht op een echt "ready"-event. (d) Midden.

4.3 — MIDDEN. Drie overlappende timer-lagen rond dezelfde gedeelde staat.

(a) 400ms parse-debounce (`werkblad.js:800-840`), 250ms cursor-poll (`:2084`), 120ms build-throttle +

pending-timer (:3163–3195) en de 500ms-retry (:2074) muteren allemaal

`atomToMathblock` / `cursorLeafMap` / statusregel. De `_buildBusy` / `_BUILD_MIN_MS` -constructie heet letterlijk "LOOP-NOODREM" (r3155) — een symptoombestrijding van de architectuur, geen ontwerp. (c) Volgorde-afhankelijke staat: bv. `cursor-poll` leest `atomToMathblock` terwijl een pending build hem net leeg heeft gezet → `cursor-info` valt stil terug op de root-mathblock (r3364–3377). (d) Midden.

4.4 — LAAG . Hint-kaders overleven een correcte LF terwijl de staat doorschuift.

(a) doLF wist alleen fout-kaders (`werkblad.js:3682` `clearFoutKaders()`); `VERANKERING.clearBoxes()` wordt bij LF nergens aangeroepen (grep r655/2015/3889/3963/3969/4173 — geen in doLF) en `kadersAan` blijft true. (c) Kaders-toggle aan → LF → groene/grijze boxen blijven op de bevroren regel staan (fixed, oude geometrie) terwijl step en boom al geëvolueerd zijn; de eerstvolgende toggle-klik "wist" alleen. (d) Laag.

5. Foutafhandeling

5.1 — MIDDEN . Stille engine-degradatie bij matcher-falen.

(a) `werkblad.js:1540–1542`: `catch(e){ dbg(...); return null; }` en op r3651 `if(pinResult == null)` `pinResult = pinpointFromPatterns(...)`. `dbg` is standaard UIT (r27–33). (c) Gooit `checkStep` (bv. door een malformed `duo_verzameling`), dan valt de hele pinpointing geruisloos terug op de oude, zwakkere tekstmatcher — de gebruiker/tester ziet niet dat hij een ander (minder betrouwbaar) oordeel krijgt. (d) Midden.

5.2 — MIDDEN . Malformed opgave: geen validatie, misleidende vervolg-feedback.

(a) `renderOpgave` r662: `expr.latex || expr.latex_display || expr.tekst || ''` — lege latex geeft één `st('er', 'Kan expressie niet evalueren')` (r679) maar rendert gewoon door; `beginUitkomst` blijft null en `resultsEqual(x, null)` (r390) is altijd false. `initTreeEngine` r879: `if(!ast) return;` — stil. (c) Ontbrekende AST/latex → elke LF meldt "Fout! Uitkomst komt niet overeen" alsof de student fout zit; de hint-knop meldt pas bij klikken 'Geen AST/node_map' (r3892). Er is geen schema-check bij laden. (d) Midden.

5.3 — LAAG / MIDDEN . Ontbrekende of foute node_map-entries worden stil gecompenseerd.

(a) `buildAtomToMathblock` r3171–3175: lege nodeMap → mapping stil leeg; de bekende `STRUCTURAL BUILD FAILED` -fail is bewust tot 1×/1,5s gedempt (r3205–3210) en faalt nog echt (STATUS.md r54). Test_harnas/README.md r41–44: het node_map-pad van A1 in 511_023 is aantoonbaar fout (`[0,0,0,0,0,0,0]`) en wordt door `locateBoundary` via de diff-omweg gemaskeerd. (c) Foute data blijft onzichtbaar in de bron hangen; elke nieuwe consument van node_map (zoals de cursor-mapping) struikelt er opnieuw over. (d) Midden voor de gemaskeerde data-fout, laag voor de demping.

6. Test-harnas

6.1 — Dekking: matcher goed, rest niet.

Wél: `checkStep`-toestandslogica over alle 26 opgaven (`batch.js` , 451 checks), gerichte repro's (`repro_b4.js` , `repro_tweeling.js` incl. adversair geval), breuk-notatie-regressienet (`regress_breuk_notatie.js` , 11 varianten + 1 `checkStep`-anker). Niet: `verankering.js` (0 tests — het meest fragiele bestand), de hele doLF/step-tracking/tree-evolutie-flow, en de **gecomponeerde pagina** (drie scripts in één namespace — waar bevinding 1.1 leeft). "451/451 groen" zegt dus niets over de browser-samenstelling.

6.2 — MIDDEN. Converter-extractie op string-markers.

(a) `regress_breuk_notatie.js:34-38`: knipt werkblad.js van `indexOf('function extractBraceContent')` tot `indexOf('// SIDEBAR')` en compileert dat blok met stubs. (c) Eén hernoemde functie of verplaatst comment → harnas test stillerjes een ander blok of crasht ("markers niet gevonden"); de claim "exact de live code" is één refactor verwijderd van onwaar. (d) Midden.

Kern-conclusie: de matcher-logica is degelijk en goed getest, maar de browser draait aantoonbaar níet de code die het harnas test (bevinding 1.1, `treesEqual`-botsing door het ontbreken van een IIFE om zowel werkblad.js als matcher.browser.js). Dat, plus de empirische pixel-fudges en shadow-DOM-afhankelijkheid in de verankering (2.1/2.2) en de setTimeout-choreografie (4.2), verklaart structureel waarom dit project al maanden achter "werkt in Node, spookt in de browser"-bugs aanjaagt. De goedkoopste high-impact fix is beide bestanden in een IIFE wikkelen (met expliciete `window.*`-exports) — dat elimineert 1.1 in één klap en maakt de globals tenminste bestand-lokaal.